Computer Networks

ASSIGNMENT 2

Name	Machani Sravani	**Register No.**	1924260126
Submission Date	04-11-2026	**Course**	Computer Networks (CSA0708)
GitHub Repository	https://github.com/MachaniSravani/CN-lab	**Nature / Duration**	Individual / 1 Week

CO2: Configure IP addressing schemes, subnetting and basic routing techniques using a simulation tool, for efficient network design and topology – packet switching, IPv4 addressing, subnetting, distance-vector and link-state routing, multicast, ICMP, IPv6, software-defined networking, network virtualisation and OpenFlow.

Assignment Title: Address, Subnet and Route a Growing Network Correctly (Total Marks – 50)

Nature of Assignment: Individual

Duration: 1 Week

Simulation Tool: Cisco Packet Tracer (with Python 3 for every “Complete the Code” program)

Bloom’s Level Assessed: K5 – Evaluate

Submission Date: 11-11-2026

Q1 – Spot the Invalid Addresses

1. Network, broadcast and host range of each device

Device	Address / prefix	Mask	Network	Broadcast	Usable range (count)	Verdict
Admission PC	172.16.45.200 /20	255.255.240.0	172.16.32.0	172.16.47.255	172.16.32.1 – 172.16.47.254 (4,094)	**Valid host.** Gateway 172.16.32.1 is inside the subnet.
Lab printer	10.0.15.255 /21	255.255.248.0	10.0.8.0	10.0.15.255	10.0.8.1 – 10.0.15.254 (2,046)	**INVALID – TRAP 1:** the address equals the broadcast address.
X-ray server	192.168.4.0 /23	255.255.254.0	192.168.4.0	192.168.5.255	192.168.4.1 – 192.168.5.254 (510)	**INVALID – TRAP 2:** the address equals the network address.
Clinic router	203.0.113.10 /29	255.255.255.248	203.0.113.8	203.0.113.15	203.0.113.9 – 203.0.113.14 (6)	**Valid** public host.

- **Method (block size):** /20 → the third octet moves in blocks of 16, so 45 lies in 32–47. /21 → blocks of 8, so 15 lies in 8–15. /23 → blocks of 2, so 4 lies in 4–5. /29 → the fourth octet moves in blocks of 8, so 10 lies in 8–15. Network = first address of the block, broadcast = last, usable hosts = $2^h - 2$.
- **Why the traps are easy to miss:** a /21 and a /23 cross the octet boundary, so ".255" and ".0" look like ordinary hosts. 10.0.15.255 looks like a broadcast only when the block 8–15 is worked out, and 192.168.4.0 is a normal-looking address but is the first address of a /23. Never judge validity from the last octet alone.
- **"Run out of addresses":** a /29 has only 8 − 2 = **6** usable addresses (one is the router), but the clinic has **18** devices – hence the NAT inquiry below.

```
q1_addresses.py - output

$ python3 q1_addresses.py
Admission PC  172.16.45.200/20  mask 255.255.240.0  net 172.16.32.0  bcast 172.16.47.255
              hosts 172.16.32.1 - 172.16.47.254 (4094 usable)  -> valid host
              gateway 172.16.32.1 inside subnet: True
Lab printer   10.0.15.255/21    mask 255.255.248.0  net 10.0.8.0  bcast 10.0.15.255
              hosts 10.0.8.1 - 10.0.15.254 (2046 usable)  -> INVALID - it is the broadcast
                  address
              gateway 10.0.8.1 inside subnet: True
X-ray server  192.168.4.0/23    mask 255.255.254.0  net 192.168.4.0  bcast 192.168.5.255
              hosts 192.168.4.1 - 192.168.5.254 (510 usable)  -> INVALID - it is the network
                   address
              gateway 192.168.4.1 inside subnet: True
Clinic router 203.0.113.10/29   mask 255.255.255.248  net 203.0.113.8  bcast 203.0.113.15
              hosts 203.0.113.9 - 203.0.113.14 (6 usable)  -> valid host
```

2. Inquiry – NAT with two PCs using source port 50000

PC	Inside local (private)	Inside global (after NAT)	Outside host
PC-1	192.168.1.11 : 50000	203.0.113.10 : 50000	198.51.100.20 : 443
PC-2	192.168.1.12 : 50000	203.0.113.10 : **50001**	198.51.100.20 : 443

- The router (NAT overload / PAT) keys each entry on the **public address and port** it allocates, not on the PC's own port. PC-2's source port 50000 is already in use on 203.0.113.10, so the router rewrites it to the next free port (50001). The two entries are different and both work at the same time.

- Return traffic: a reply to 203.0.113.10:50000 is rewritten to 192.168.1.11:50000, and a reply to 203.0.113.10:50001 to 192.168.1.12:50000. Each public address offers about 64,000 ports, so even one address can serve all 18 devices; 6 usable addresses give roughly 380,000 translations.

Datagram switching vs virtual circuits

- **Why packets of one file arrive out of order (datagram):** every packet is routed independently by a table look-up. If two equal-cost paths exist (for example 20 ms and 30 ms) or the route changes mid-transfer, packet 2 can take the slower path and arrive after packet 3. TCP restores the order with sequence numbers.

Feature	Datagram (IP)	Virtual circuit (ATM, Frame Relay, MPLS LSP)
Set-up	None – send immediately	Call set-up first (about one round trip of delay)
Header	Full destination address (IPv4 header 20 B)	Short circuit identifier (a few bytes)
Path	Each packet may take a different path	One fixed path for the whole call
Order	May arrive out of order	Always in order
Failure	Routes around the failure automatically	Circuit breaks; it must be set up again
Switch state	No per-flow state	One table entry per circuit
Quality of service	Hard to guarantee	Bandwidth and delay can be reserved

Alternative Concept Analysis – a public /27 instead of NAT behind a /29

Measure	NAT behind the /29	Public /27 (no NAT)
Public addresses needed	1 (router) – 6 usable in the /29 are enough	18 devices + router = **19**; a /28 has only 14 usable, so a **/27 (30 usable)** is the smallest block
Address block size / cost	8 addresses	32 addresses = 4× the block, so about 4× the leasing cost
Exposure to the internet	Inbound connections blocked by default; devices hidden	Every device is directly reachable – needs a firewall, patching and ACLs on every host
Application problems	Breaks end-to-end addressing, inbound servers need port-forwarding, protocols that carry IP addresses in the payload (active FTP, SIP/VoIP, IPsec AH) need helpers, port exhaustion, no per-device address in logs	**None of these** – true end-to-end connectivity, simple servers, VoIP and VPN work, each device is traceable
Verdict	Cheaper and safer by default – suitable for a clinic that only browses and uses cloud systems	Justified only if the clinic hosts inbound services; then add a stateful firewall

Q2 – Subnet a Campus That Keeps Growing

1. Design with equal-size subnets (192.168.10.0/24, four subnets)

Four subnets need 2 extra bits, so every subnet is a /26 (255.255.255.192) with 64 − 2 = **62** hosts.

Subnet	Network / prefix	Mask	First host	Last host	Broadcast	Needed	Result
Labs	192.168.10.0 /26	255.255.255.192	.1	.62	.63	100	**FAILS** – 38 hosts short
Staff	192.168.10.64 /26	255.255.255.192	.65	.126	.127	50	Fits (12 wasted)
Library	192.168.10.128 /26	255.255.255.192	.129	.190	.191	25	Fits (37 wasted)
WAN link	192.168.10.192 /26	255.255.255.192	.193	.254	.255	2	Fits (**60 wasted**)

- Equal-size subnets cannot work here: a /25 would hold the 100 laboratory hosts but the /24 would give only **two** subnets, and a /26 gives four subnets but the laboratories do not fit. The point-to-point link wastes 60 addresses.

2. Design with VLSM (largest first)

Subnet	Hosts + 2	Block	Network / prefix	Mask	First host	Last host	Broadcast
Labs	102	128	192.168.10.0 /25	255.255.255.128	192.168.10.1	192.168.10.126	192.168.10.127
Staff	52	64	192.168.10.128 /26	255.255.255.192	192.168.10.129	192.168.10.190	192.168.10.191
Library	27	32	192.168.10.192 /27	255.255.255.224	192.168.10.193	192.168.10.222	192.168.10.223
WAN link	4	4	192.168.10.224 /30	255.255.255.252	192.168.10.225	192.168.10.226	192.168.10.227

- Block = the smallest power of two ≥ hosts + 2; prefix = 32 − $\log_2$(block). Allocated 228 of 256 addresses; the next free address is **192.168.10.228** (28 addresses left).

3. The trap – a 60-host hostel six months later

- 60 hosts + 2 = 62, so the hostel needs a **/26 block of 64 addresses** that starts on a multiple of 64 (.0, .64, .128 or .192). All four are taken. The free space is only 192.168.10.228/30 + .232/29 + .240/28 = 28 addresses, and its largest aligned block is a /28 (14 hosts). **It cannot be fitted without renumbering.**
- Total demand is now 128 + 64 + 64 + 32 + 4 = **292 addresses**, which is more than the 256 in a /24 – so even a complete renumbering inside this /24 is impossible.
- **Smallest change that works:** give the hostel a second block, **192.168.11.0/26**, and advertise 192.168.10.0/23 (the /24 plus the next /24) as one summary. No existing subnet is renumbered.

```
q2_summary_hostel.py - output

$ python3 q2_summary_hostel.py
free blocks after VLSM plan: ['192.168.10.228/30', '192.168.10.232/29', '192.168.10.240/28']
     = 28 addresses
hostel needs a /26 (64 addresses); largest free block = 192.168.10.240/28 -> fits: False
total demand with hostel = 292 > 256 addresses in a /24

Equal-size split of the /24 into 4 subnets: ['192.168.10.0/26', '192.168.10.64/26',
    '192.168.10.128/26', '192.168.10.192/26'] -> 62 hosts each; Labs (100) does NOT fit

Summarisation
summary of 8-11: [IPv4Network('172.16.8.0/22')]
172.16.8.0/21 covers 172.16.8.0 - 172.16.15.255 ; wrongly covers 172.16.13.0/24 -> True
correct with 172.16.12.0/24: [IPv4Network('172.16.8.0/22'), IPv4Network('172.16.12.0/24')]
```

4. Route summarisation

Network	3rd octet (binary)
172.16.8.0/24	0000 1000
172.16.9.0/24	0000 1001
172.16.10.0/24	0000 1010
172.16.11.0/24	0000 1011

- The first 6 bits of the third octet (000010) are common, so the prefix is 16 + 6 = 22: the single summary is **172.16.8.0/22** (covers 172.16.8.0 – 172.16.11.255).
- **What goes wrong with 172.16.8.0/21:** a /21 covers 172.16.8.0 – 172.16.15.255, i.e. eight /24s. Only 8–12 belong to the campus, so the router now advertises 172.16.13.0, .14.0 and .15.0 that it does not own. Traffic for those networks (which may belong to another site) is pulled to this router and dropped or looped, and the real owner is blocked by the less-specific route. Correct fix: advertise 172.16.8.0/22 and 172.16.12.0/24 separately (or reserve 13–15 for this campus first).

Complete the Code – TODO 1

```
# TODO 1 (completed)
block = 1 << math.ceil(math.log2(hosts + 2))
prefix = 32 - int(math.log2(block))
ipaddress.ip_network(f"{to_ip(addr)}/{prefix}")   # raises ValueError if misaligned
print(f"{name:9s}{to_ip(addr)}/{prefix} hosts {to_ip(addr+1)} - {to_ip(addr+block-2)}
bcast {to_ip(addr+block-1)}")
addr += block
```

q2_vlsm.py - completed code

```
"""Variable-length subnetting (VLSM) of 192.168.10.0/24 for a college campus."""
import math, ipaddress
def to_int(ip):
    a, b, c, d = map(int, ip.split("."))
    return (a << 24) | (b << 16) | (c << 8) | d
def to_ip(n):
    return ".".join(str((n >> s) & 255) for s in (24, 16, 8, 0))
def vlsm(base, needs):
    addr = to_int(base)
    for name, hosts in sorted(needs, key=lambda x: -x[1]):
        block = 1 << math.ceil(math.log2(hosts + 2))
        prefix = 32 - int(math.log2(block))
        net = ipaddress.ip_network(f"{to_ip(addr)}/{prefix}")   # raises if misaligned
        print(f"{name:9s}{to_ip(addr)}/{prefix} hosts {to_ip(addr+1)} -
            {to_ip(addr+block-2)} bcast {to_ip(addr+block-1)}")
        addr += block
    return to_ip(addr)
needs = [("Labs", 100), ("Staff", 50), ("Library", 25), ("WAN link", 2)]
print("next free address:", vlsm("192.168.10.0", needs))
```

q2_vlsm.py - output

```
$ python3 q2_vlsm.py
Labs     192.168.10.0/25 hosts 192.168.10.1 - 192.168.10.126 bcast 192.168.10.127
Staff    192.168.10.128/26 hosts 192.168.10.129 - 192.168.10.190 bcast 192.168.10.191
Library  192.168.10.192/27 hosts 192.168.10.193 - 192.168.10.222 bcast 192.168.10.223
WAN link 192.168.10.224/30 hosts 192.168.10.225 - 192.168.10.226 bcast 192.168.10.227
next free address: 192.168.10.228
```

Packet Tracer verification (router CAMPUS-R1)

```
interface GigabitEthernet0/0
 ip address 192.168.10.1 255.255.255.128      ! Labs
interface GigabitEthernet0/1
 ip address 192.168.10.129 255.255.255.192    ! Staff
interface GigabitEthernet0/2
 ip address 192.168.10.193 255.255.255.224    ! Library
interface Serial0/0/0
 ip address 192.168.10.225 255.255.255.252    ! WAN link
 clock rate 64000
! hostel attempt (rejected by IOS - overlaps Library/WAN):
interface GigabitEthernet0/3
 ip address 192.168.10.229 255.255.255.192
```

Alternative Concept Analysis – allocating the smallest subnet first

- Starting with the WAN link: 192.168.10.0/30; the next free address is .4, so the Library /27 is placed at 192.168.10.4/27 – but a /27 must start on a multiple of 32. Its real network is 192.168.10.0/27 and it **overlaps** the WAN link. Staff (.36/26) and Labs (.100/25) are misaligned in the same way (their real networks are .0/26 and .0/25).
- Forcing every block onto its own boundary repairs the overlap but wastes space: WAN .0/30, Library .32/27, Staff .64/26, Labs .128/25 leaves the **28 addresses .4 – .31 unusable and fragmented**, so the plan has no contiguous room at all.
- **Why VLSM assigns largest first:** block sizes are powers of two and every block must start at a multiple of its own size. After the largest block, the running address is a multiple of every smaller size, so each following block is automatically aligned and no gaps appear.

q2_vlsm_smallest_first.py - output

```
$ python3 q2_vlsm_smallest_first.py
WAN link 192.168.10.0/30  OK (aligned)
Library  192.168.10.4/27  MISALIGNED -> real network 192.168.10.0/27 (overlaps earlier
    blocks)
Staff    192.168.10.36/26  MISALIGNED -> real network 192.168.10.0/26 (overlaps earlier
    blocks)
Labs     192.168.10.100/25  MISALIGNED -> real network 192.168.10.0/25 (overlaps earlier
    blocks)
cursor ends at 192.168.10.228

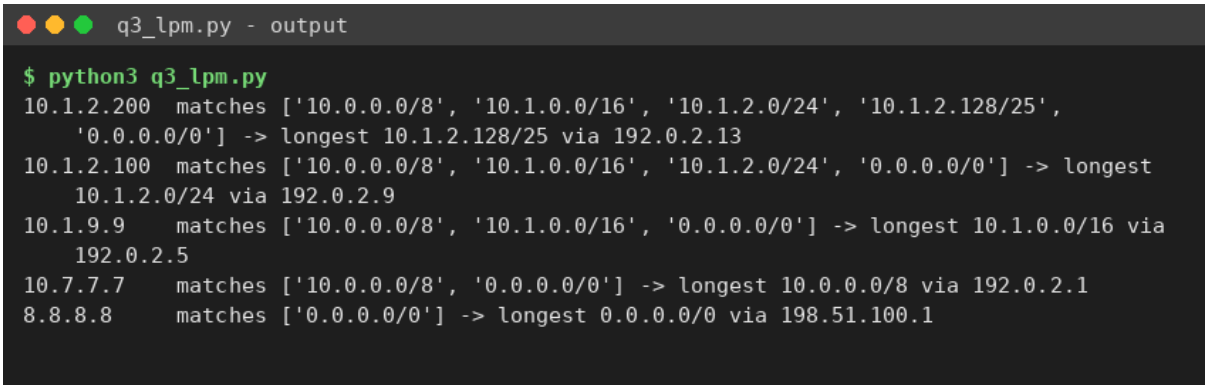
If blocks are forced onto aligned boundaries (padding):
WAN link 192.168.10.0/30
Library  192.168.10.32/27
Staff    192.168.10.64/26
Labs     192.168.10.128/25
gap wasted: 28 addresses (192.168.10.4 - 192.168.10.31)
```

Q3 – Which Way Will the Packet Go?

1. Next hop for each destination

Destination	Matching routes	Chosen (longest prefix)	Next hop
10.1.2.200	/8, /16, /24, /25, default	10.1.2.128/25 (covers .128–.255)	**192.0.2.13**
10.1.2.100	/8, /16, /24, default	10.1.2.0/24 (.100 is outside the /25)	**192.0.2.9**
10.1.9.9	/8, /16, default	10.1.0.0/16	**192.0.2.5**
10.7.7.7	/8, default	10.0.0.0/8	**192.0.2.1**
8.8.8.8	default only	0.0.0.0/0	**198.51.100.1**

- **Rule used – longest-prefix match:** of all routes whose network contains the destination, the router chooses the one with the longest prefix (most specific). The order of the entries does not matter, and the default route 0.0.0.0/0 is used only when nothing else matches.



2. Complete the Code – TODO 1 and TODO 2 (R1)

```
enable
configure terminal
hostname R1
interface GigabitEthernet0/0
 ip address 192.168.10.1 255.255.255.0
 no shutdown
! TODO 1 (completed)
interface Serial0/0/0
 ip address 10.0.0.1 255.255.255.252
 clock rate 64000
 no shutdown
exit
! TODO 2 (completed)
ip route 192.168.20.0 255.255.255.0 10.0.0.2
end
write memory
show ip route
```

R2 (reverse route):

```
hostname R2
interface GigabitEthernet0/0
 ip address 192.168.20.1 255.255.255.0
 no shutdown
interface Serial0/0/0
 ip address 10.0.0.2 255.255.255.252     ! DCE clock is on R1 only
 no shutdown
exit
ip route 192.168.10.0 255.255.255.0 10.0.0.1
```

- **Expected result:** show ip route on R1 lists S 192.168.20.0/24 [1/0] via 10.0.0.2, and a main-campus PC can ping a city-campus PC.

- **Removing the clock rate on the DCE end:** with no clocking the serial line cannot synchronise, so the interface shows “up, line protocol is down”; the static route is withdrawn from the routing table (its next hop is no longer reachable) and pings fail with “Destination host unreachable”. Re-entering clock rate 64000 restores the link.

3. Backup link with a floating static route (administrative distance 200)

Add a second serial link 10.0.1.0/30 (R1 S0/0/1 = 10.0.1.1, R2 S0/0/1 = 10.0.1.2) and a backup route with a higher distance than the main route (static = 1):

```
! R1
ip route 192.168.20.0 255.255.255.0 10.0.0.2          ! main, AD 1
ip route 192.168.20.0 255.255.255.0 10.0.1.2 200      ! floating backup, AD 200
! R2
ip route 192.168.10.0 255.255.255.0 10.0.0.1
ip route 192.168.10.0 255.255.255.0 10.0.1.1 200
```

Alternative Concept Analysis – OSPF instead of static routes

```
router ospf 1
 network 192.168.10.0 0.0.0.255 area 0
 network 10.0.0.0 0.0.0.3 area 0
 network 10.0.1.0 0.0.0.3 area 0
```

Campuses (one LAN each)	Static routes (main only / with floating backup)	OSPF (lines in total)
2 campuses, main + backup link	2 / **4** lines	**8** lines (router ospf + 3 network lines on each router)
5 campuses in a ring (each router: LAN + 2 links)	4 routes per router = 20 / **40** lines (two routes for every remote LAN)	**20** lines (4 per router, independent of the number of remote LANs)

- Static routing is cheaper for 2 campuses, but its lines grow with the **square** of the number of sites; OSPF grows only with the number of local interfaces, so it wins from about 3 campuses upward.
- **Recovery (typical defaults):** a floating static route reacts at once when the local interface goes down (about a second), but it **never notices a failure beyond the next hop** while the link stays up – traffic is black-holed until someone intervenes. OSPF detects a silent failure after the dead interval (40 s with default hello 10 s) and then recomputes in about 1–2 s; with an interface-down event it reconverges in a few seconds, and fast hellos can cut detection below a second.

Q4 – Watch Distance-Vector Routing Count to Infinity

1. Round-by-round distance tables (links A–B 2, A–C 7, B–C 3, B–D 8, C–D 1, D–E 4)

Each cell is the vector of distances to (A, B, C, D, E); ∞ = no route known yet.

Round	A	B	C	D	E
0 (neighbours only)	0 2 7 ∞ ∞	2 0 3 8 ∞	7 3 0 1 ∞	∞ 8 1 0 4	∞ ∞ ∞ 4 0
1	0 2 **5** **8** ∞	2 0 3 **4** **12**	**5** 3 0 1 **5**	**8** **4** 1 0 4	∞ **12** **5** **4** 0
2	0 2 5 **6** **12**	2 0 3 4 **8**	5 3 0 1 5	**6** 4 1 0 4	**12** **8** 5 4 0
3	0 2 5 6 **10**	2 0 3 4 8	5 3 0 1 5	6 4 1 0 4	**10** 8 5 4 0
4 (no change)	0 2 5 6 10	2 0 3 4 8	5 3 0 1 5	6 4 1 0 4	10 8 5 4 0

- **Converged after 4 rounds** (rounds 1–3 change something; round 4 changes nothing, which proves convergence). Bold = value changed in that round.
- **Why A's best route to C is not the direct link:** the direct link costs 7, but A → B → C costs 2 + 3 = **5**, so the next hop for C is B. In the same way A reaches D via B (cost 6) and E via B (cost 10).

Complete the Code – TODO 1

```
# TODO 1 (completed)
if link[u, n] + old[n][dest] < table[u][dest]:
    table[u][dest] = link[u, n] + old[n][dest]
    changed = True
```

q4_dv.py - completed code

```
"""Distance-vector routing (Bellman-Ford exchange) among five campus routers."""
INF = 99
names = "ABCDE"
cost = {("A", "B"): 2, ("A", "C"): 7, ("B", "C"): 3, ("B", "D"): 8,
        ("C", "D"): 1, ("D", "E"): 4}
link = {}
for (u, v), c in cost.items():
    link[u, v] = link[v, u] = c
table = {u: {v: (0 if u == v else link.get((u, v), INF)) for v in names} for u in names}
rounds = 0
changed = True
while changed:
    changed = False
    rounds += 1
    old = {u: dict(table[u]) for u in names}      # vectors received this round
    for u in names:
        for n in names:
            if (u, n) not in link:
                continue
            for dest in names:
                # TODO 1 (completed): Bellman-Ford update through neighbour n
                if link[u, n] + old[n][dest] < table[u][dest]:
                    table[u][dest] = link[u, n] + old[n][dest]
                    changed = True
print(f"converged after {rounds} rounds")
for u in names:
    print(u, " ".join(f"{table[u][v]:2d}" for v in names))
```

```
q4_dv.py - output

$ python3 q4_dv.py
converged after 4 rounds
A  0  2  5  6 10
B  2  0  3  4  8
C  5  3  0  1  5
D  6  4  1  0  4
E 10  8  5  4  0
```

After the C–D link fails (cost set to 99) – q4_dv_fail.py:

Router	to A	to B	to C	to D	to E
A	0	2	5	**10**	**14**
B	2	0	3	**8**	**12**
C	5	3	0	**11**	**15**
D	**10**	**8**	**11**	0	4
E	**14**	**12**	**15**	4	0

- **Converged after 3 rounds.** Every path to D and E now goes B → D (cost 8); for example C → D rises from 1 to 11 (C → B → D = 3 + 8) and A → D from 6 to 10. Bold = values that changed from the earlier table.

```
q4_dv_fail.py - output

$ python3 q4_dv_fail.py
converged after 3 rounds
A  0  2  5 10 14
B  2  0  3  8 12
C  5  3  0 11 15
D 10  8 11  0  4
E 14 12 15  4  0
```

2. Count to infinity on X – Y – Z (cost 1 each), link Y–Z fails

Round	X → Z (via Y)	Y → Z	What happens
0 (before)	2	1	Normal: Y reaches Z directly, X through Y
1	2	**3**	Y loses Z, hears X's old "Z = 2" and believes X has a path: 2 + 1 = 3 (through X!)
2	**4**	3	X hears Y's "Z = 3" and updates to 4
3	4	**5**	Y hears X's 4 and updates to 5
4 … 14	6, 8, 10, 12, 14, 16	5, 7, 9, 11, 13, 15	Distances climb by 2 per pair of rounds (a routing loop X ↔ Y)
15	16	**16**	RIP infinity = 16 reached – Z finally declared unreachable after **15 rounds**

- **Split horizon with poison reverse:** X learned Z **from Y**, so X advertises "Z = ∞ (16)" back to Y instead of 2. Y therefore never believes X has a path to Z: Y sets Z = ∞ in round 1 and X follows in round 2 – the loop is prevented and Z is unreachable after **2 rounds instead of 15**.

q4_count_to_infinity.py - output

```
$ python3 q4_count_to_infinity.py
Without split horizon:
  round  0 (before failure): X->Z =  2  Y->Z =  1
  round  1: X->Z =  2  Y->Z =  3
  round  2: X->Z =  4  Y->Z =  3
  round  3: X->Z =  4  Y->Z =  5
  round  4: X->Z =  6  Y->Z =  5
  round  5: X->Z =  6  Y->Z =  7
  round  6: X->Z =  8  Y->Z =  7
  round  7: X->Z =  8  Y->Z =  9
  round  8: X->Z = 10  Y->Z =  9
  round  9: X->Z = 10  Y->Z = 11
  round 10: X->Z = 12  Y->Z = 11
  round 11: X->Z = 12  Y->Z = 13
  round 12: X->Z = 14  Y->Z = 13
  round 13: X->Z = 14  Y->Z = 15
  round 14: X->Z = 16  Y->Z = 15
  round 15: X->Z = 16  Y->Z = 16
  -> Z declared unreachable after 15 rounds
With split horizon + poison reverse:
  round  0 (before failure): X->Z =  2  Y->Z =  1
  round  1: X->Z =  2  Y->Z = 16
  round  2: X->Z = 16  Y->Z = 16
  -> Z declared unreachable after 2 rounds
```

3. RIP in Packet Tracer

```
router rip
 version 2
 no auto-summary
 network 192.168.1.0
 network 10.0.12.0
 passive-interface GigabitEthernet0/0
! verify
show ip route rip
show ip protocols
debug ip rip
```

- Test: ping across the network, shut down one link, watch show ip route rip. RIP default timers: update 30 s, invalid 180 s, hold-down 180 s, flush 240 s. When the interface itself goes down, a triggered update with metric 16 is sent at once; a silent failure is only noticed after the 180 s invalid timer – the slow convergence that makes RIP unsuitable for large networks.

Alternative Concept Analysis – link-state routing on the same network

Measure (C–D fails)	Distance vector	Link-state
Messages	Every router sends its vector to each neighbour every round: 2 × 6 links = 12 messages per round × 3 rounds = **36 vector messages** (5 entries each)	C and D each flood one LSA over the 5 surviving links: 2L − (N − 1) = 10 − 4 = 6 transmissions each → **12 LSA messages** (one link entry each)
Rounds until all routes are correct	**3 rounds**	**3 flooding hops** (C's LSA reaches E over C–B–D–E), then every router runs Dijkstra locally – no further messages
Count to infinity	Possible (needs split horizon, poison reverse, hold-down)	Not possible – every router holds the same complete map
Cost	Light CPU and memory	Full topology database and Dijkstra at every router

.

Q5 – Why Does OSPF Treat 1 Gbps and 10 Gbps the Same?

1. OSPF cost = reference bandwidth ÷ link bandwidth (minimum 1)

Link	Cost, reference 100 Mbps (default)	Cost, reference 100,000 Mbps (100 Gbps)
10 Mbps	100 ÷ 10 = **10**	100,000 ÷ 10 = **10,000**
100 Mbps	100 ÷ 100 = **1**	100,000 ÷ 100 = **1,000**
1 Gbps	100 ÷ 1,000 = 0.1 → rounded up to **1**	100,000 ÷ 1,000 = **100**
10 Gbps	100 ÷ 10,000 = 0.01 → rounded up to **1**	100,000 ÷ 10,000 = **10**

- **The problem:** with the default 100 Mbps reference, 100 Mbps, 1 Gbps and 10 Gbps links all cost **1**. OSPF cannot see that the 10 Gbps path is ten times faster; when hop counts are equal the paths tie and traffic is shared (ECMP) or the 1 Gbps path is used, so the slower link congests.
- **New reference bandwidth: 100,000 Mbps (auto-cost reference-bandwidth 100000).** It makes 10 Gbps cost 10 and leaves room for 100 Gbps links (cost 1). It must be configured identically on **every** router, otherwise they calculate different costs.

q5_ospf.py - output (1/2)

```
$ python3 q5_ospf.py
reference bandwidth 100 Mbps:
   10 Mbps   cost = max(1, 100/10) = 10
   100 Mbps  cost = max(1, 100/100) = 1
   1 Gbps    cost = max(1, 100/1000) = 1
   10 Gbps   cost = max(1, 100/10000) = 1
reference bandwidth 100000 Mbps:
   10 Mbps   cost = max(1, 100000/10) = 10000
   100 Mbps  cost = max(1, 100000/100) = 1000
   1 Gbps    cost = max(1, 100000/1000) = 100
   10 Gbps   cost = max(1, 100000/10000) = 10

Dijkstra from R1, reference 100 Mbps:
   (tie) R5 also reachable at cost 2 via R4 -> equal-cost paths
   (tie) R3 also reachable at cost 2 via R4 -> equal-cost paths
   (tie) R6 also reachable at cost 3 via R5 -> equal-cost paths
   R1 -> R1: cost    0  path R1
   R1 -> R2: cost    1  path R1 > R2
   R1 -> R3: cost    2  path R1 > R2 > R3
   R1 -> R4: cost    1  path R1 > R4
   R1 -> R5: cost    2  path R1 > R2 > R5
   R1 -> R6: cost    3  path R1 > R2 > R3 > R6
```

2. Dijkstra by hand from R1 (6-router ISP of my design)

Link	Speed	Cost (ref 100 Gbps)
R1–R2, R2–R3, R3–R6	10 Gbps	10
R1–R4, R4–R5, R5–R6, R2–R5	1 Gbps	100
R3–R4	100 Mbps	1,000

Step	Router settled (cost)	Updates (distance to R2 / R3 / R4 / R5 / R6)
0	R1 (0)	R2 = 10, R4 = 100; others ∞

Link		Speed	Cost (ref 100 Gbps)
1	R2 (10)	R3 = 20, R5 = 10 + 100 = 110	
2	R3 (20)	R6 = 20 + 10 = 30 (R4 via R3 = 1,020 – worse than 100)	
3	R6 (30)	R5 via R6 = 130 – worse than 110, no change	
4	R4 (100)	R5 via R4 = 200 – worse, no change	
5	R5 (110)	All routers settled	

- **Shortest-path tree from R1:** R1 → R2 (10), R2 → R3 (20), R3 → R6 (**30**, all-10 Gbps path), R2 → R5 (110), R1 → R4 (100). With the default reference, R1 → R6 via the 10 Gbps path (cost 3) ties with R1–R4–R5–R6 over 1 Gbps links (cost 3); now it is 30 against 300, so only the 10 Gbps path is used.

q5_ospf.py - output (2/2)

```
Dijkstra from R1, reference 100000 Mbps:
   pick R1 (cost 0)  settled=['R1']
      relax R1->R2: R2 = 10
      relax R1->R4: R4 = 100
   pick R2 (cost 10)  settled=['R1', 'R2']
      relax R2->R3: R3 = 20
      relax R2->R5: R5 = 110
   pick R3 (cost 20)  settled=['R1', 'R2', 'R3']
      relax R3->R6: R6 = 30
   pick R6 (cost 30)  settled=['R1', 'R2', 'R3', 'R6']
   pick R4 (cost 100)  settled=['R1', 'R2', 'R3', 'R4', 'R6']
   pick R5 (cost 110)  settled=['R1', 'R2', 'R3', 'R4', 'R5', 'R6']
   R1 -> R1: cost    0  path R1
   R1 -> R2: cost   10  path R1 > R2
   R1 -> R3: cost   20  path R1 > R2 > R3
   R1 -> R4: cost  100  path R1 > R4
   R1 -> R5: cost  110  path R1 > R2 > R5
   R1 -> R6: cost   30  path R1 > R2 > R3 > R6
```

3. Packet Tracer configuration

```
interface GigabitEthernet0/0
 bandwidth 10000000        ! emulate 10 Gbps (kbit/s)
interface GigabitEthernet0/1
 bandwidth 1000000         ! 1 Gbps
router ospf 1
 router-id 1.1.1.1
 auto-cost reference-bandwidth 100000
 network 10.0.12.0 0.0.0.3 area 0
! verify
show ip ospf interface brief      ! cost 10 on 10 Gbps, 100 on 1 Gbps links
show ip route ospf                ! R6 LAN [110/30] via the 10 Gbps path
```

4. One OSPF area or several?

- **Decision: a single backbone area 0 for this 6-router design.** The link-state database is tiny, Dijkstra takes microseconds, and one area has no ABRs, no inter-area suboptimal routing and the simplest operation.
- **Move to several areas** (area 0 as the backbone plus one area per city cluster) once the provider grows beyond roughly 30–50 routers or unstable links cause frequent SPF runs: LSAs and SPF

are then confined to one area, and the ABRs can summarise each city's prefixes. The trade-off is more design effort and every area must attach to area 0.

Alternative Concept Analysis – EIGRP or IS-IS

EIGRP metric = 256 × (10^7 ÷ minimum bandwidth in kbit/s + sum of delays in tens of µs). Assumed paths of 3 links with the usual 10 µs delay on each Gigabit/10-Gigabit interface (3 × 1 = 3 units).

Protocol	Path of 1 Gbps links	Path of 10 Gbps links	Distinguishes them without a change?
OSPF, default reference (100 Mbps)	1 + 1 + 1 = 3	1 + 1 + 1 = 3	**No** – tie
OSPF, reference 100 Gbps	100 × 3 = 300	10 × 3 = 30	Yes, after the change
EIGRP (classic metric)	256 × (10^7 ÷ 10^6 + 3) = 256 × 13 = **3,328**	256 × (10^7 ÷ 10^7 + 3) = 256 × 4 = **1,024**	**Yes** – bandwidth is part of the metric
IS-IS (default metric 10 per link on common implementations)	10 × 3 = 30	10 × 3 = 30	**No** – needs manual metrics

- **Decision:** EIGRP avoids the problem with no change, but it is Cisco-originated and its classic bandwidth term saturates at 10 Gbps. OSPF with a one-line reference-bandwidth change is multi-vendor and scales to 100 Gbps, so it remains the better choice for a state ISP unless the whole network is Cisco.

Q6 – IPv6, ICMP, Multicast and SDN in a Smart City

1. IPv6 shortening and EUI-64

- 2001:0db8:0000:0000:0000:ff00:0042:8329 → remove leading zeros in each group, then replace the longest run of all-zero groups by "::" → **2001:db8::ff00:42:8329**.
- MAC 00:1A:2B:3C:4D:5E: split it in two (00:1A:2B | 3C:4D:5E), insert **FF:FE** in the middle and **flip the universal/local bit** of the first byte (0x00 → 0x02): interface identifier **021A:2BFF:FE3C:4D5E**, so the link-local address is fe80::21a:2bff:fe3c:4d5e.

2. How traceroute uses ICMP Time Exceeded

- traceroute sends probes with TTL = 1; the first router decrements the TTL to 0, discards the packet and returns **ICMP Time Exceeded (type 11)** whose source address is that router. The next probe has TTL = 2 and exposes the second router, and so on. When a probe finally reaches the destination, the destination answers (ICMP echo reply or port unreachable) and the trace ends. The round-trip time of each reply gives the per-hop delay. (IPv6 uses the Hop Limit field and ICMPv6 type 3.)

3. Multicast MAC for 239.1.2.3

- The multicast MAC is 01:00:5E followed by the lower 23 bits of the group address: 239.1.2.3 → lower 23 bits = 1.2.3 → **01:00:5E:01:02:03**. (32 IPv4 groups share each MAC because 5 bits are dropped.)
- **One stream, several control rooms:** each control room joins group 239.1.2.3 with IGMP and the routers build a distribution tree (PIM). The camera sends **one** copy; routers and switches copy the packet only where the paths to receivers split. Example: a 4 Mbps stream to 5 control rooms costs the camera 4 Mbps with multicast but 5 × 4 = 20 Mbps with unicast.

4. OpenFlow rules and the SDN decision

Packet	Rules matched	Winning rule (highest priority)	Action
HTTP to 10.5.1.1 (TCP 80)	priority 200 (dst 10.5.0.0/16) and priority 100 (TCP 80)	Priority **200**	**Forward out port 3** – not sent to the controller
SSH to 10.6.1.1 (TCP 22)	only priority 0	Priority **0** (table-miss)	**Dropped**

- **Judgement – adopt SDN, in stages.** Benefits: one central view and policy for 2 million sensors and the camera network, routing changes in minutes by software, easy traffic engineering and automation. Risks: the controller is a single point of failure and a security target (use a clustered controller), flow-table size limits (TCAM), and "send to controller" rules can overload it. Recommended: pilot SDN on the camera/sensor segments with a redundant controller cluster, keep an ordinary routing protocol (OSPF) as a fallback underlay, then extend to the whole city.

q6_ipv6.py - output

```
$ python3 q6_ipv6.py
short form : 2001:db8::ff00:42:8329
EUI-64 ID  : 021a:2bff:fe3c:4d5e  -> link-local fe80::21a:2bff:fe3c:4d5e
multicast MAC: 01:00:5e:01:02:03
OpenFlow: {'dst': '10.5.1.1', 'tcp': 80} -> priority 200: output port 3
OpenFlow: {'dst': '10.6.1.1', 'tcp': 22} -> priority 0: drop
6in4 on 100-byte packet: 120 bytes (+20 B = 20% extra)
2 million sensors x 1 packet/s x 20 B = 320.0 Mbps extra
```

Alternative Concept Analysis – IPv6 only, dual-stack or 6in4 tunnelling

Approach	Advantages	Disadvantages
IPv6 only	Simplest network, huge address space (2 million sensors need $\geq 2^{21}$ IPv4 addresses, an impractical public block)	Old IPv4 devices and services unreachable without NAT64/DNS64
Dual-stack	Works with both worlds, no translation, no tunnel overhead	Two sets of configuration and security policy; more memory
6in4 tunnel (IPv6 inside IPv4, protocol 41)	Crosses IPv4-only links without upgrading them	+20 B per packet, lower MTU (1,480 B), harder troubleshooting, tunnel end-points become bottlenecks

- **Tunnel overhead:** a 100-byte sensor packet becomes 100 + 20 = **120 bytes** (+20 %). If each of the 2 million sensors sends one packet per second, the tunnel adds 2,000,000 × 20 B × 8 = **320 Mbps** of pure overhead.
- **Decision:** start with **dual-stack** in the core and edge so existing IPv4 equipment keeps working, deploy all new sensor networks IPv6-only (with NAT64 where needed), and use 6in4 tunnels only as a temporary bridge across IPv4-only links.

Q7 – Find the Weak Point, Redesign and Validate

Network chosen: the engineering-college campus of Q2 (192.168.10.0/24 with VLSM), which fails when the hostel is added. The redesign is an addressing and summarisation change – no extra equipment is added.

A. Identify the limitation

B. The VLSM plan is correct for today's needs, but it uses 228 of 256 addresses and leaves **28 addresses** in three small pieces (192.168.10.228/30, .232/29, .240/28). The largest aligned block is a /28 (14 hosts), while the hostel needs a /26 (64 addresses): **the hostel allocation fails** (q7 test 1).

- The root cause is not the VLSM method but the **fixed single /24 with no reserved growth space**: total demand is 128 + 64 + 64 + 32 + 4 = **292 addresses > 256**, so no rearrangement or renumbering inside the /24 can succeed. The four separate prefixes also need four routes upstream.

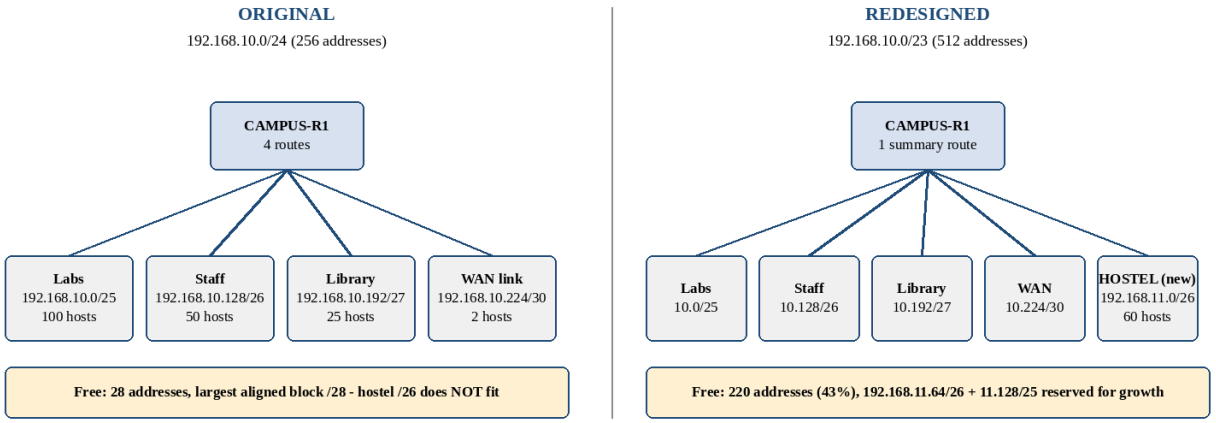
B. Design objective

#	Measurable objective	Original
1	Place the 60-host hostel with **0 existing subnets renumbered**	Fails
2	Keep **≥ 40 % of the address space free** after the hostel for growth	11 % free (28 of 256)
3	Advertise the whole campus upstream as **1 summary route**	4 specific routes

C. The revised network

Keep every existing subnet exactly as it is and **extend the campus block from 192.168.10.0/24 to 192.168.10.0/23** (512 addresses). The hostel gets 192.168.11.0/26; 192.168.11.64/26 and 192.168.11.128/25 are reserved for growth. The campus router advertises **192.168.10.0/23** as one summary and holds a discard route (Null0) for the summary.

- **New trade-offs:** a second /24 of private space is consumed (free, but it must be reserved in the college's plan); the summary also covers 220 unused addresses, so traffic to unassigned addresses must be discarded by the Null0 route to avoid loops; the hostel needs one more router interface (or a VLAN sub-interface on the existing one).



Subnet	Network / prefix	Mask	Usable hosts	Gateway (router)	Status
Labs	192.168.10.0 /25	255.255.255.128	192.168.10.1 – .126	192.168.10.1	Unchanged
Staff	192.168.10.128 /26	255.255.255.192	.129 – .190	192.168.10.129	Unchanged

Subnet	Network / prefix	Mask	Usable hosts	Gateway (router)	Status
Library	192.168.10.192 /27	255.255.255.224	.193 – .222	192.168.10.193	Unchanged
WAN link	192.168.10.224 /30	255.255.255.252	.225 – .226	192.168.10.225	Unchanged
Hostel	**192.168.11.0 /26**	255.255.255.192	192.168.11.1 – .62	192.168.11.1	**New**
Reserved	192.168.11.64 /26 and 192.168.11.128 /25	—	190 addresses	—	Growth

```
interface GigabitEthernet0/3
 description Hostel
 ip address 192.168.11.1 255.255.255.192
 no shutdown
ip route 192.168.10.0 255.255.254.0 Null0         ! discard unused part of the summary
! upstream (city) router
ip route 192.168.10.0 255.255.254.0 192.168.10.226
```

D. Prediction before implementing

The hostel allocation will **succeed with 0 renumbered subnets**; free space will rise from **28 addresses (11 %)** to **220 addresses (43 %)** after the hostel is added; and the upstream router will need **1 summary route** instead of 4 specific routes.

E. Implement and validate – Original → Redesigned

Test	Original (/24)	Redesigned (/23)
1. Allocate 60-host hostel	**FAILED** – no aligned /26 free	Allocated 192.168.11.0/26
2. Overlaps between subnets	None	None
3. Largest free block	192.168.10.240/28	192.168.11.128/25
4. Free addresses	28 of 256 (11 %)	220 of 512 (43 %)
5. Subnets renumbered	0	0
6. Summary route	192.168.10.0/24 (replaces 4 routes)	192.168.10.0/23 (replaces 5 routes)
7. Summary covers unused space	28 addresses	220 addresses (Null0 discard)

q7_validate.py - output

```
$ python3 q7_validate.py
== ORIGINAL  (single /24) ==  pool 192.168.10.0/24 (256 addresses)
  test 1 hostel (60 hosts) allocated : FAILED - no aligned /26 free
  test 2 overlaps among subnets      : False
  test 3 largest free block          : 192.168.10.240/28
  test 4 free addresses              : 28 of 256 (11%)
  test 5 subnets renumbered          : 0
  test 6 single summary route        : 192.168.10.0/24 (replaces 4 specific routes)
  test 7 summary covers unused space  : 28 addresses (discard with Null0)

== REDESIGNED (/23, hostel in 192.168.11.0/26) ==  pool 192.168.10.0/23 (512 addresses)
  test 1 hostel (60 hosts) allocated : 192.168.11.0/26
  test 2 overlaps among subnets      : False
  test 3 largest free block          : 192.168.11.128/25
  test 4 free addresses              : 220 of 512 (43%)
  test 5 subnets renumbered          : 0
  test 6 single summary route        : 192.168.10.0/23 (replaces 5 specific routes)
  test 7 summary covers unused space  : 220 addresses (discard with Null0)
```

Packet Tracer tests (same for both versions): ping from a Labs PC to the Staff PC, the Library PC, the WAN-side router and (redesigned only) the Hostel PC; show ip route on the upstream router; attempt to configure the hostel interface.

F. Defend the final design

- **Objective achieved:** hostel placed with 0 renumbered subnets (objective 1), 43 % free ≥ 40 % (objective 2), and one summary route (objective 3) – all three measured by q7_validate.py.
- **New costs:** a second /24 of private space is reserved; the summary includes 220 unused addresses, so a Null0 discard route is mandatory; one extra router interface; the hostel is a separate broadcast domain.
- **Where the original is still preferable:** if the hostel is never built, the single /24 is simpler and uses 89 % of its space; and if the college only has one /24 (for example a public block from the provider), the redesign is impossible and the hostel would need its own provider block or NAT behind a separate network.